```
  def index
    @projects = Project.all
  end

  def create
    @workflow = CreatesProject.new(
      name: params[:project][:name],
      task_string: params[:project][:tasks])
    @workflow.create
    redirect_to projects_path
  end
end
```

And now you have an action without a view, resulting in the error message that starts with ActionView::MissingTemplate: Missing template projects/index,.... To get past this error, create a blank view file at app/views/projects/index.html.erb.

Now the error is on the following line of the test:

```
adding projects allows a user to create a project with tasks
     Failure/Error: expect(page).to have_content("Project Runway")
       expected to find text "Project Runway" in ""
 # ./spec/system/add_project_spec.rb:11:in `block (2 levels) in <top (required)>'
```

At this point, the Capybara test is evaluating the HTML response from your application. You're asking the test to have_content, which is defined by Capybara and which passes if the response either literally contains a string argument or matches a regular-expression argument. Specifically, you're asserting that the output contains the string Project Runway (which is the name of the newly created project) and the string 8 (which is its size).

This is a weak test. There are all kinds of ways, for example, that the number 8 could appear in the response HTML. To pick one unlikely scenario, the user could be journalist and author Jennifer 8. Lee.[3] The struggle when view-testing is to find a balance between a test that validates something meaningful about the output and one that isn't so tied to the markup that it will break when a designer looks at the page cross-eyed.

Let's make the test pass first, then explore how you can strengthen the test. Passing the test requires a straightforward Rails view:

3. http://en.wikipedia.org/wiki/Jennifer_8._Lee